

ChainBreak

Attack-Path-Aware Containment for Security Incidents

AI Defense Lab 2026

Track 1: Cyber Defense & Threat Response

Synthetic Enterprise Environment | Simulation Only

TEAM NO: 77

TEAMMATES: SIRI PAIDIPALLI AND BHARADWAJ KATNA

Project Overview

ChainBreak is an attack-path-aware security decision system designed to help defenders choose containment actions during an active security incident.

Traditional incident response may identify several possible containment actions, such as disabling a compromised account, isolating a workstation, revoking credentials, or disabling a service. While multiple actions may stop an attacker, they can have very different operational impacts.

ChainBreak focuses on a specific question:

What is the smallest safe containment action that breaks all viable attacker paths to the protected asset while disrupting the least legitimate activity?

The system reconstructs viable attacker paths using a synthetic enterprise graph and security telemetry, correlates supporting evidence, assesses incident risk, evaluates possible containment actions, recommends the lowest-impact complete containment option, requires explicit analyst approval, simulates the response, and independently verifies whether containment succeeded.

Threat and Protected Asset

The demonstration models an enterprise incident in which the identity **Alice** has been compromised. The protected asset is the **Customer Database**.

The attacker can reach the database through two independent routes:

Attack Path 1

Alice → Alice-Laptop → Dev-Server → Deployment Token → Deployment Service → Customer Database

Attack Path 2

Alice → Alice-Laptop → Dev-Server → Backup Service Token → Backup Service → Customer Database

Both paths share the compromised identity, workstation, and development server before separating into independent credential and service routes.

This creates an important containment problem. Removing one route does not necessarily contain the incident because the attacker may still reach the protected asset through the alternate route.

Security Signals and Evidence

ChainBreak uses synthetic enterprise telemetry created specifically for the project.

The primary incident contains nine synthetic security events representing activity including:

- Unusual successful authentication
- Suspicious PowerShell execution
- SSH activity
- Credential-file access
- Token authentication
- Database-access attempts

Events are correlated with entities and relationships in the attack graph.

A graph transition is considered supported only when the observed telemetry matches the relevant source entity, target entity, and expected relationship. This allows ChainBreak to show not only that an attack path is theoretically possible, but also what evidence supports the reconstructed path.

For the primary demonstration scenario:

Viable Attack Paths: 2

Risk Score: 100/100

Severity: CRITICAL

Evidence Confidence: 100/100

Priority: P1

Evidence confidence represents the completeness of telemetry supporting the reconstructed attack paths. It does not represent the probability that an attack occurred.

ChainBreak Security Workflow

ChainBreak implements the following end-to-end security workflow:

Synthetic Enterprise Telemetry

↓

Evidence Correlation

↓

Attack-Path Reconstruction

↓

Risk + Evidence Assessment

↓

Containment Analysis

↓

Minimum-Impact Recommendation

↓

Human Approval

↓

Simulated Containment

↓

Attack-Graph Recalculation

↓

Containment Verification

↓

Audit Trail

The architecture is intentionally centered around the security decision rather than unnecessary system complexity.

Each stage contributes directly to answering three questions:

1. **Can the attacker reach the protected asset?**
2. **What should be contained?**
3. **Did the containment actually work?**

Attack-Path Reconstruction

ChainBreak models identities, infrastructure, credentials, services, and protected assets as a directed graph. The graph engine searches for viable simple paths between the compromised identity and the protected asset. For the primary incident, ChainBreak discovers two viable attack paths. This is important because security incidents may contain alternate routes. A response that removes one attacker path can appear successful while another route remains available. The attack graph therefore becomes the basis for both containment analysis and post-action verification.

Risk and Evidence Assessment

ChainBreak uses deterministic and explainable security logic rather than an opaque risk prediction.

The risk assessment considers factors including:

- Criticality and reachability of the protected asset
- Number of viable attacker paths
- Evidence of credential-file access
- Evidence coverage across the reconstructed attack paths

The primary attack scenario reaches a risk score of **100/100**, corresponding to **CRITICAL severity and P1 priority**.

ChainBreak also provides MITRE ATT&CK context for supported evidence, including:

- **T1078 – Valid Accounts**
- **T1059.001 – PowerShell**
- **T1552.001 – Credentials In Files**
- **T1021.004 – SSH**

ATT&CK mappings provide additional investigation context. They are not treated as independent proof that malicious activity occurred.

Containment Analysis

After reconstructing the attack paths, ChainBreak evaluates possible containment actions and their configured operational-disruption costs.

Containment Action	Operational Cost	Paths Remaining	Result
Revoke Deployment Token	1	1	INCOMPLETE
Revoke Backup Service Token	2	1	INCOMPLETE
Revoke Both Tokens	3	0	COMPLETE
Disable Alice Account	5	0	COMPLETE
Isolate Alice-Laptop	8	0	COMPLETE
Disable Deployment Service	10	1	INCOMPLETE
Disable Backup Service	12	1	INCOMPLETE
Isolate Dev-Server	20	0	COMPLETE

Several actions can completely contain the incident.

For example, disabling Alice, isolating Alice-Laptop, or isolating Dev-Server would remove the attacker's access to the Customer Database. However, these actions have higher configured operational impact.

ChainBreak evaluates individual containment actions and combinations of actions and identifies:

Recommended Containment

Revoke Deployment Token + Revoke Backup Service Token

Operational Cost: 3

Paths Before: 2

Predicted Paths After: 0

The recommendation is therefore the lowest-cost action set that achieves complete containment across the available candidate actions. The operational costs used in this prototype are illustrative disruption units. They are not monetary values or measured production business impact.

Human-in-the-Loop Response

ChainBreak does not automatically execute disruptive containment actions. After generating a recommendation, the system requires explicit analyst approval.

The workflow follows:

Recommendation → Human Approval → Simulated Action

For the demonstration, the analyst identity is represented as **SOC-Analyst**. All containment actions are simulated. ChainBreak does not disable real accounts, revoke real credentials, isolate real endpoints, or modify external infrastructure. The containment operation is performed on a copy of the attack graph, preserving the original environment.

Recommendation Is Not Verification

A major design principle of ChainBreak is that recommending a security action does not prove that containment succeeded. After an approved action is simulated, ChainBreak independently recalculates the attack graph.

Partial Containment

If only the Deployment Token is revoked:

2 Paths → 1 Path

The Deployment Service route is removed, but the attacker can still reach the Customer Database through:

Alice → Alice-Laptop → Dev-Server → Backup Service Token → Backup Service → Customer Database

ChainBreak therefore reports:

CONTAINMENT INCOMPLETE

Recommended Containment

If both compromised tokens are revoked:

2 Paths → 0 Paths

The protected asset is no longer reachable from the compromised identity.

ChainBreak reports:

CONTAINMENT VERIFIED

This separation between predicted containment and independently verified containment prevents the system from assuming that a recommended action successfully eliminated every attacker route.

Auditability

ChainBreak maintains an in-memory audit trail of important security decisions and actions.

The audit records events including:

- Incident analysis

- Recommendation generation
- Analyst approval or denial
- Blocked execution when approval is absent
- Simulated containment actions
- Post-action verification

Audit entries include timestamps, relevant identifiers, status, and supporting details.

The current audit mechanism exists for the lifetime of the workflow and is not intended to represent a persistent enterprise audit platform.

Evaluation

ChainBreak was evaluated against multiple security conditions rather than only the primary attack scenario.

Evaluation Results

Scenario	Result
Normal Activity	PASS
Active Compromise	PASS
Suspicious but Non-Reachable	PASS
Missing Telemetry	PASS
Partial Containment	PASS
Recommended Containment	PASS
High-Impact Alternatives	PASS

Overall Result

7 / 7 Evaluation Scenarios Passed

The scenarios evaluate different aspects of the system:

Normal Activity

The protected asset is not reachable and no disruptive containment is recommended.

Active Compromise

Two viable paths exist. After approved recommended containment, the result changes from **2** → **0 paths** and containment is verified.

Suspicious but Non-Reachable

Suspicious evidence exists, but there is no viable path to the protected asset. ChainBreak does not recommend unnecessary disruptive containment.

Missing Telemetry

Some supporting telemetry is removed. ChainBreak continues operating but reduces evidence coverage to **50%** and confidence to **50/100**, exposing uncertainty instead of treating incomplete evidence as complete.

Partial Containment

Only one credential is revoked. The result changes from **2** → **1 paths**, and ChainBreak identifies the surviving Backup Service route.

Recommended Containment

Both tokens are revoked after approval. The result changes from **2** → **0 paths**, producing **CONTAINMENT VERIFIED**.

High-Impact Alternatives

ChainBreak compares the recommended cost-3 action set with more disruptive alternatives such as disabling Alice or isolating infrastructure.

Reliability and Testing

ChainBreak includes **58 automated tests**.

The test suite covers:

- Attack-graph traversal
- Multiple attack paths
- Cycles and duplicate relationships
- Containment optimization
- Deterministic tie breaking
- Graph immutability
- Protected-asset safety
- Evidence correlation
- Missing telemetry
- Risk and confidence behavior
- Analyst approval enforcement
- Partial containment
- Independent verification
- Audit behavior
- Evaluation determinism

The automated verification runner reports:

OVERALL: PASS

Security and Safety Controls

ChainBreak includes multiple controls appropriate to a defensive security system.

Human Approval

Potentially disruptive containment does not execute without explicit analyst approval.

Simulation-Only Response

No real account, credential, endpoint, service, or protected asset is modified.

Protected Asset Safety

The protected asset cannot be proposed as a containment target.

Graph Integrity

Containment is performed on a copy of the graph. The original environment remains unchanged.

Evidence-Based Decisions

Containment recommendations are based on graph reachability and configured operational impact rather than an unexplained model output.

Auditability

Analysis, decisions, approvals, simulated actions, and verification results are recorded.

Synthetic Data

The project does not require confidential, private, or unauthorized enterprise data.

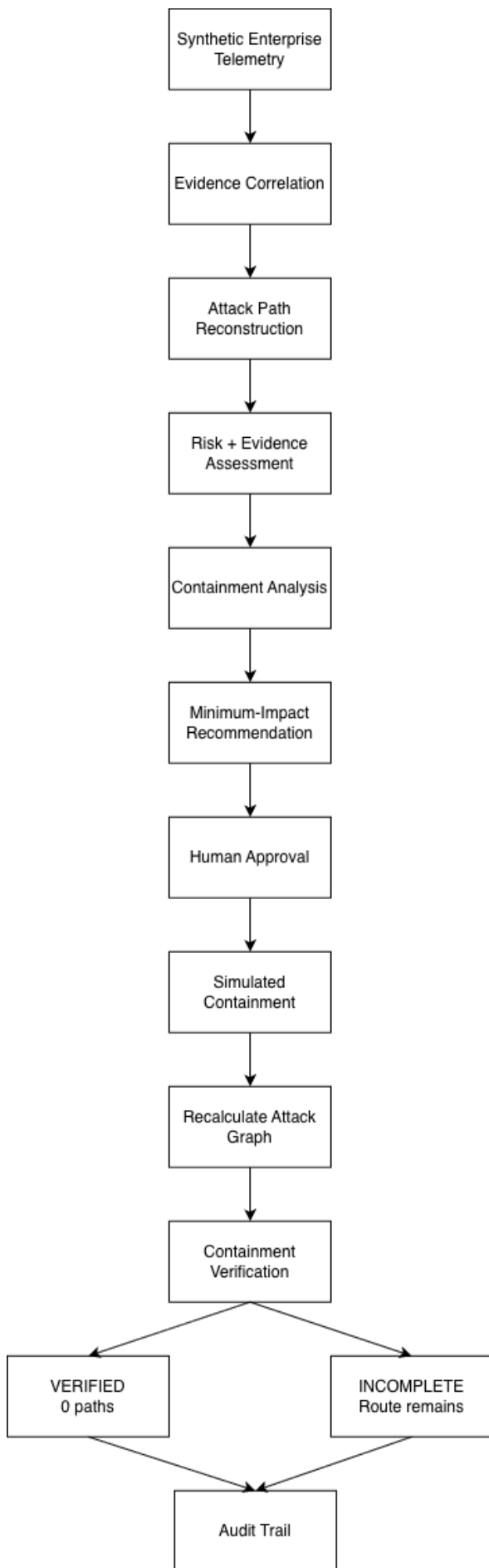
Architecture and Operational Readiness

ChainBreak separates the security workflow into reusable backend components for:

- Environment and graph modeling
- Attack-path discovery
- Incident evidence correlation
- Risk and confidence assessment
- Containment optimization
- Approval workflow
- Simulated execution
- Independent verification
- Evaluation
- Audit logging

The Streamlit dashboard and command-line workflows use the same backend security logic.

The current implementation prioritizes explainability, reproducibility, safety, and evaluation over unnecessary architectural complexity.



Production-Readiness Limitations

ChainBreak is a working security prototype, not a production incident-response platform.

Important limitations include:

- The enterprise environment and telemetry are synthetic.
- Operational disruption costs are configured illustrative values rather than measured business impact.
- Containment actions are simulated.
- Graph relationships are supplied by the synthetic environment rather than automatically discovered from live infrastructure.
- The prototype does not model persistence or credentials already stolen outside the represented graph.
- Evidence correlation is designed for the controlled incident environment and does not establish causality across arbitrary enterprise telemetry.
- Exhaustive attack-path and containment-subset analysis is designed for the current small graph and has not been validated at enterprise scale.
- Evaluation scenarios are synthetic regression tests rather than a production detection-accuracy benchmark.
- ChainBreak currently has no live SIEM, EDR, identity-provider, cloud-security, or enforcement integrations.

A production implementation would require live telemetry ingestion, enterprise asset and identity discovery, calibrated business-impact modeling, persistent audit storage, authentication and authorization, scalable graph analysis, and integrations with security response platforms.

Running ChainBreak

Install Dependencies

```
python3 -m pip install -r requirements.txt
```

Launch the Application

```
python3 -m streamlit run app.py
```

Open the local Streamlit URL shown in the terminal.

Run Evaluation

```
python3 -m src.evaluation
```

Run Automated Verification

```
python3 scripts/verify.py
```

Run All Tests

```
python3 -m unittest discover -s tests -v
```

Dependencies, Data, and APIs

Dependencies

ChainBreak uses the following direct runtime/UI dependencies:

- **Streamlit 1.49.1** for the dashboard and interactive workflow.
- **Graphviz Python package 0.21** for graph visualization.

Dependencies are declared in `requirements.txt`.

Data

The enterprise environment, incident events, identities, credentials, services, attack relationships, and evaluation scenarios used by ChainBreak are synthetic and were created for the project.

No confidential or unauthorized enterprise data is used.

APIs and External Services

ChainBreak does not require external security APIs or proprietary security services for its runtime workflow.

Containment is simulated locally and no external response actions are executed.

Hackathon Disclosure

ChainBreak was developed for **AI Defense Lab 2026**.

The project uses synthetic project data and the open-source/runtime dependencies declared in `requirements.txt`.

Conclusion

ChainBreak demonstrates that incident containment should not stop at selecting an action.

A security response must answer:

Can the attacker still reach the protected asset?

By combining evidence-supported attack-path reconstruction with minimum-impact containment analysis, explicit human approval, and independent post-action verification, ChainBreak provides a controlled and explainable way to move from security signals to a defensible containment outcome.